

EVALUASI TENGAH SEMESTER (ETS)

Algoritma dan Pemrograman

Program Studi Teknik Instrumentasi
Semester Genap 2025/2026

PROJECT BASED LEARNING (PBL)

Sistem Instrumentasi Kontrol Pompa Otomatis
Berbasis Rust

Dosen Pengampu	: Ahmad Radhy, S.Si., M.Si.
Durasi Pengerjaan	: 1 Pekan
Bahasa Pemrograman	: Rust
Metode Penilaian	: Project + Ujian Lisan

Kelompok : 2

Mahasiswa 1 : Hafidzan Yaafi Az Zaahir

NRP : 2042251062

Mahasiswa 2 : Higmatiar Maulana Sakti

NRP : 2042251128

Departemen Teknik Instrumentasi
Institut Teknologi Sepuluh Nopember

Daftar Isi

1	Pendahuluan	2
2	Studi Kasus Instrumentasi	2
3	Algoritma dan Flowchart	3
3.1	Penjelasan Flowchart	3
4	Konsep Pemrograman Berbasis Objek (OOP)	5
5	Implementasi Komputasi Numerik	6
6	Penjelasan Program Rust	7
7	Struktur Program Sistem Kontrol Pompa	9
8	Hasil Pengujian	10
9	Analisis Hasil	10
10	Kesimpulan	11

1 Pendahuluan

Sistem kontrol otomatis pada dasarnya merupakan penerapan dari ilmu teknik kendali yang menggabungkan berbagai komponen elektronika seperti sensor, aktuator, mikrokontroler, dan relay dalam satu sistem yang terintegrasi dan bekerja secara sinergis. Prinsip dasar dari sistem ini adalah memanfaatkan umpan balik (feedback) dari sensor untuk mengambil keputusan secara otomatis apakah pompa perlu dihidupkan atau dimatikan berdasarkan kondisi level air yang terdeteksi. Dengan prinsip kerja ini, sistem kontrol pompa otomatis mampu menjaga level air dalam tangki penampungan pada rentang yang diinginkan secara konsisten tanpa memerlukan pengawasan manusia secara terus-menerus.

Dari sisi ekonomi dan efisiensi energi, sistem kontrol pompa otomatis juga memberikan manfaat yang sangat signifikan. Dengan kemampuan sistem untuk secara otomatis menghidupkan pompa hanya ketika dibutuhkan dan mematikannya segera setelah kondisi yang diinginkan tercapai, konsumsi energi listrik dapat ditekan secara optimal. Hal ini tidak hanya berdampak pada penghematan biaya operasional, tetapi juga berkontribusi pada upaya efisiensi energi yang saat ini menjadi perhatian global mengingat semakin terbatasnya sumber energi dan semakin meningkatnya kesadaran akan pentingnya penggunaan energi secara bijak dan bertanggung jawab.

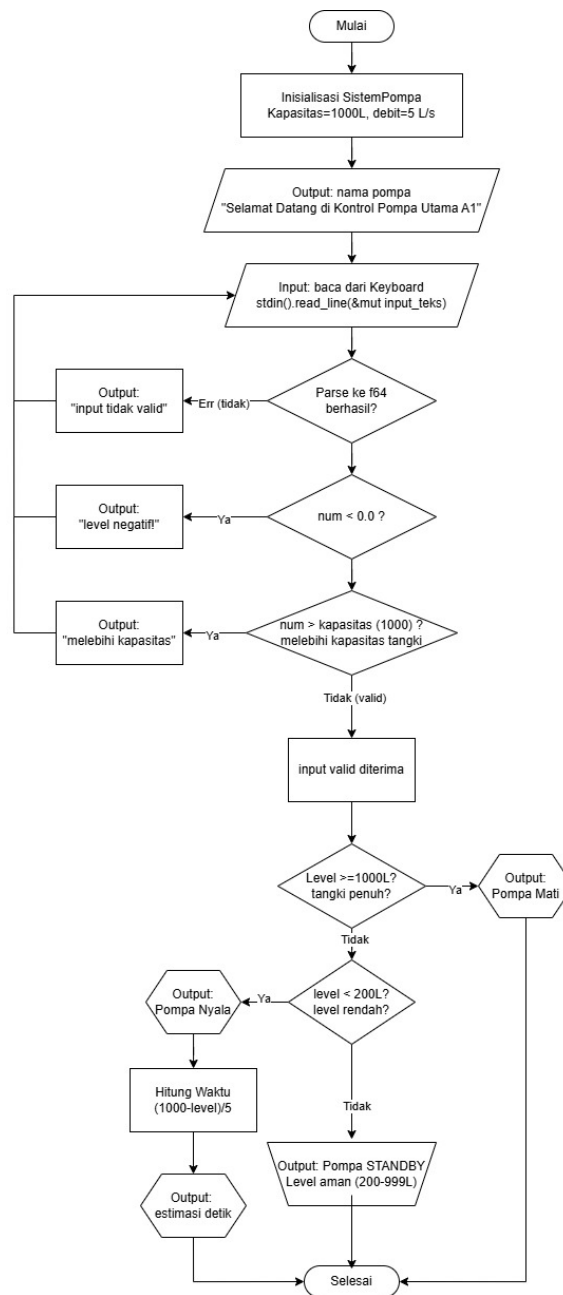
Berdasarkan berbagai pertimbangan di atas, pengembangan dan implementasi sistem kontrol pompa otomatis menjadi topik yang sangat relevan dan penting untuk dikaji secara mendalam, baik dari sisi perancangan perangkat keras maupun perangkat lunak yang mendasarinya. Penelitian dan pengembangan di bidang ini terus berkembang seiring dengan hadirnya komponen-komponen elektronika yang semakin canggih, terjangkau, dan mudah diintegrasikan, sehingga membuka peluang untuk menciptakan sistem kontrol pompa yang semakin handal, efisien, dan mudah digunakan oleh masyarakat luas.

2 Studi Kasus Instrumentasi

Studi kasus yang diambil adalah monitoring tangki penyimpanan air. Tantangan utama adalah ketidakpastian nilai sensor (adanya *error bias*). Sistem harus mampu:

1. Melakukan kalibrasi mandiri untuk menghilangkan selisih pembacaan sensor dengan nilai aktual.
2. Mengendalikan operasi pompa secara otomatis
3. Melakukan aksi *safety interlocking*:
 - **Low Low (LL):** Mengaktifkan pompa jika kurang dari batas minimal air .
 - **High High (HH):** Mematikan pompa jika sudah sampai batas maksimal ketinggian air.

3 Algoritma dan Flowchart



Gambar 1: Caption

3.1 Penjelasan Flowchart

Berdasarkan flowchart pada Gambar 1, berikut adalah penjelasan mengenai alur logika sistem kontrol pompa:

1. Terminal (Start/End)

- **Mulai (Start):** Menandai awal eksekusi program. Di dalam kode, ini setara dengan pemanggilan fungsi `main()`.

- **Selesai (Stop):** Menandai akhir dari proses kontrol. Program akan berhenti setelah menampilkan status pompa dan estimasi waktu.

2. Inisialisasi & Input (Paralelogram)

- **Inisialisasi SistemPompa:** Program membuat objek `pompa_industri` dengan parameter tetap: kapasitas 1000L, debit 5L/detik, dan ambang batas rendah 200L.
- **Input Level Air:** Operator memasukkan data level cairan saat ini melalui fungsi `minta_input_angka` yang divalidasi agar tidak negatif atau melebihi kapasitas.

3. Struktur Keputusan / Decision (Belah Ketupat) Merupakan inti dari logika *safety interlocking* pada sistem:

- **Level $\geq$ Kapasitas?:** Mengecek apakah tangki sudah penuh. Jika **Ya**, pompa dimatikan (OFF) untuk mencegah *overflow*.
- **Level < Batas Rendah?:** Mengecek apakah level di bawah 200L. Jika **Ya**, pompa dinyalakan (ON) untuk mengisi tangki.
- **Kondisi Lain (Else):** Jika level berada di antara 200L dan 1000L, pompa tetap dalam status **Standby** (Mati).

4. Proses & Perhitungan (Persegi Panjang)

- **Hitung Estimasi Waktu:** Jika pompa menyala, sistem menjalankan komputasi numerik menggunakan metode `hitung_estimasi_waktu` dengan rumus $t = \frac{\Delta V}{Q}$.
- **Set Status Pompa:** Mengubah variabel status operasional berdasarkan keputusan logika di atas.

5. Output (Paralelogram)

- **Tampilkan Status & Waktu:** Program mencetak hasil akhir ke terminal, memberikan informasi status pompa dan durasi pengisian hingga penuh.

4 Konsep Pemrograman Berbasis Objek (OOP)

1. **Pengertian Object Oriented Programming (OOP)** Object Oriented Programming (OOP) atau pemrograman berbasis objek merupakan paradigma pemrograman yang menggunakan objek sebagai dasar utama dalam pengembangan program. Objek digunakan untuk merepresentasikan data serta fungsi yang saling berhubungan di dalam suatu sistem. Konsep OOP bertujuan untuk mempermudah pengelolaan program agar lebih terstruktur, mudah dikembangkan, mudah dipelihara, dan dapat digunakan kembali. Pada project sistem monitoring level tangki ini, konsep OOP diterapkan menggunakan bahasa pemrograman Rust dengan memanfaatkan struct dan implementation block (impl). Struktur program dibuat agar data tangki dan seluruh proses monitoring berada di dalam satu objek sehingga sistem menjadi lebih modular dan efisien.
2. **Penerapan OOP pada Sistem kontrol pompa** Pada program sistem kontrol pompa, konsep OOP digunakan untuk mengelompokkan seluruh parameter dan fungsi kontrol ke dalam objek kontrol. Objek ini menyimpan data kapasitas tangki, batas Low Low (LL), batas High High (HH), dan pembacaan sensor level. Berikut merupakan deklarasi struct yang digunakan pada program:

```
1 let pompa_industri = SistemPompa {  
2     nama_pompa: String::from("Pompa Utama A1"),  
3     kapasitas_tangki: 1000.0,  
4     debit_pompa: 5.0,  
5     struct SistemPompa {  
6         nama_pompa: String,  
7         kapasitas_tangki: f64,  
8         debit_pompa: f64,  
9     };
```

Struct tersebut berfungsi sebagai blueprint atau rancangan objek yang menyimpan seluruh data monitoring tangki.

3. **konsep OOP Object** adalah Konsep dalam OOP pada sistem kontrol pompa adalah proses membungkus data dan fungsi dalam satu kesatuan objek sehingga akses terhadap data tersebut dapat dikontrol dengan baik. Dalam program Rust yang dibuat, enkapsulasi diterapkan melalui penggunaan struct SistemPompa yang menyimpan seluruh atribut penting seperti nama pompa, kapasitas tangki, dan debit pompa. Data-data ini tidak diakses secara langsung dari luar, melainkan digunakan melalui method seperti `hitungestimasiwaktu()`, yang bertugas mengolah data internal untuk menghasilkan informasi berupa estimasi waktu pengisian tangki. Dengan pendekatan ini, logika perhitungan menjadi terpusat dan lebih terorganisir,

5 Implementasi Komputasi Numerik

Komputasi numerik digunakan untuk mengolah data mentah sensor:

- **Koreksi Linear:** Menggunakan penambahan bias secara konstan untuk mendapatkan nilai riil.
- **Konversi Skala:** Melakukan pemetaan (*mapping*) nilai linear dari domain meter ($0 - h$) ke domain volume ($0 - V_{max}$).

Persamaan Matematis yang Digunakan

Program menggunakan metode perhitungan linear untuk menentukan durasi waktu yang dibutuhkan fluida untuk memenuhi kapasitas tangki. Persamaan dasarnya adalah:

$$t = \frac{V_{kapasitas} - V_{level}}{Q_{debit}} \quad (1)$$

Di mana:

- t : Estimasi waktu (detik).
- $V_{kapasitas}$: Kapasitas maksimal tangki (liter).
- V_{level} : Level cairan saat ini (liter).
- Q_{debit} : Laju alir atau debit pompa (liter/detik).

Implementasi dalam Kode (Rust)

Secara teknis, logika ini diletakkan di dalam *method* `hitung_estimasi_waktu` pada blok implementasi `struct SistemPompa`:

```
1 fn hitung_estimasi_waktu(&self, level_sekarang: f64) -> f64 {
2     let sisa_volume = self.kapasitas_tangki - level_sekarang;
3     if sisa_volume <= 0.0 {
4         0.0
5     } else {
6         sisa_volume / self.debit_pompa
7     }
8 }
```

Karakteristik Komputasi Numerik dalam Program

- **Presisi Data:** Menggunakan tipe data `f64` untuk memastikan tingkat presisi yang tinggi.
- **Koreksi Kondisi Batas:** Pengecekan `if sisa_volume <= 0.0` untuk mencegah ralat logika.

- **Deterministik:** Memberikan hasil yang pasti berdasarkan parameter input.
- **Output Terformat:** Format `{:.1}` untuk memudahkan pembacaan hasil oleh operator.

6 Penjelasan Program Rust

Source code yang dibuat merupakan implementasi sistem kontrol pompa sederhana berbasis bahasa Rust yang berjalan secara interaktif melalui terminal. Program diawali dengan fungsi `mintainputangka()` yang bertugas membaca input dari pengguna dan mengonversinya menjadi tipe data numerik (`f64`). Fungsi ini juga dilengkapi dengan validasi sederhana untuk menangani kesalahan input dengan memberikan nilai default jika input tidak valid. Selanjutnya, digunakan konsep pemrograman berbasis objek melalui struct `SistemPompa` yang berfungsi sebagai representasi sistem pompa dengan atribut berupa nama pompa, kapasitas tangki, dan debit pompa. Pada bagian `impl`, didefinisikan method `hitungestimasiwaktu()` yang digunakan untuk menghitung estimasi waktu yang dibutuhkan hingga tangki penuh berdasarkan selisih volume dan debit pompa.

Kode berikut dirancang untuk berjalan di terminal secara interaktif:

```

1 use std::io; // Library untuk input/output
2
3 fn minta_input_angka(kapasitas: f64) -> f64 {
4     loop {
5         let mut input_teks = String::new();
6         io::stdin().read_line(&mut input_teks).expect("Gagal
membaca input");
7         match input_teks.trim().parse::<f64>() {
8             Ok(num) => {
9                 if num < 0.0 {
10                     println!("Level tidak boleh negatif!");
11                 } else if num > kapasitas {
12                     println!("Level tidak boleh melebihi
kapasitas tangki ({:.0} liter)!", kapasitas);
13                 } else {
14                     return num;
15                 }
16             },
17             Err(_) => println!("Input tidak valid, coba lagi
:"),
18         }
19     }
20 }
21
22 // Poin 3: Pemrograman Berbasis Objek (Struct)
23 struct SistemPompa {
24     nama_pompa: String,
25     batas_level_rendah: f64,
26     kapasitas_tangki: f64, // Liter

```



```

27     debit_pompa: f64,
28 }
29
30 impl SistemPompa {
31     // Poin 4: Komputasi Numerik
32     // Fungsi untuk menghitung sisa waktu sampai penuh
33     fn hitung_estimasi_waktu(&self, level_sekarang: f64) ->
34     f64 {
35         let sisa_volume = self.kapasitas_tangki -
36         level_sekarang;
37         if sisa_volume <= 0.0 {
38             0.0
39         } else {
40             sisa_volume / self.debit_pompa
41         }
42     }
43 }
44
45 fn main() {
46     // Inisialisasi objek
47     let pompa_industri = SistemPompa {
48         nama_pompa: String::from("Pompa Utama A1"),
49         kapasitas_tangki: 1000.0,
50         debit_pompa: 5.0,
51         batas_level_rendah: 200.0,
52     };
53
54     println!("--- Selamat Datang di Kontrol {} ---",
55     pompa_industri.nama_pompa);
56
57     // Meminta input level air
58     println!("Masukkan level air saat ini (liter): ");
59     let level_input = minta_input_angka(pompa_industri.
60     kapasitas_tangki);
61
62     // Logika Kontrol (Poin 2: Implementasi Algoritma)
63     if level_input >= pompa_industri.kapasitas_tangki {
64         println!("Status: TANGKI PENUH. Pompa MATI.");
65     } else if level_input < pompa_industri.batas_level_rendah
66     {
67         println!("Status: LEVEL RENDAH. Pompa NYALA.");
68         let waktu = pompa_industri.hitung_estimasi_waktu(
69         level_input);
70         println!("Estimasi tangki penuh dalam: {:.1} detik",
71         waktu);
72     } else {
73         println!("Status: LEVEL AMAN. Pompa STANDBY.");
74     }
75 }

```

7 Struktur Program Sistem Kontrol Pompa

- **Import Library**

- Menggunakan `use std::io;`
- Berfungsi untuk menangani input dan output (I/O) dari pengguna.

- **Fungsi Input (`minta_input_angka`)**

- Membaca input dari pengguna dalam bentuk teks.
- Mengubah (parse) input menjadi tipe data `f64`.
- Menggunakan `trim()` untuk menghapus spasi atau enter.
- Memiliki penanganan error:
 - * Jika input bukan angka, maka menggunakan nilai default `0.0`.

- **Deklarasi Struct (OOP)**

- `struct SistemPompa` sebagai representasi objek pompa.
- Memiliki atribut:
 - * `nama_pompa` (String)
 - * `kapasitas_tangki` (liter)
 - * `debit_pompa` (liter/detik)

- **Implementasi Method**

- Method: `hitung_estimasi_waktu(&self, level_sekarang: f64)`
- Fungsi:
 - * Menghitung sisa volume tangki.
 - * Jika tangki penuh, maka hasil = 0.
 - * Jika belum penuh, maka waktu = sisa volume dibagi debit pompa.
- Menerapkan konsep enkapsulasi dan komputasi numerik.

- **Fungsi Utama (`main`)**

- Titik awal eksekusi program.
- Inisialisasi objek `pompa_industri`.
- Menampilkan informasi awal kepada pengguna.
- Meminta input level air saat ini.

- **Logika Kontrol (Algoritma)**

- Menggunakan percabangan **if-else**:
 - * Jika level $\geq$ kapasitas tangki:
 - Status: Tangki penuh
 - Pompa mati
 - * Jika level < 200 liter:
 - Status: Level rendah
 - Pompa nyala
 - Menghitung estimasi waktu pengisian
 - * Selain itu:
 - Status: Aman
 - Pompa standby

• Output Program

- Menampilkan status kondisi tangki.
- Menampilkan estimasi waktu jika pompa aktif.

8 Hasil Pengujian

Pengujian dilakukan untuk memvalidasi logika *interlocking* dan fungsi proteksi input.

Tabel 1: Hasil Pengujian Sistem Kontrol Pompa dengan Garis Lengkap

Kondisi	Input (L)	Ekspektasi Output	Status
Level Rendah	150.0	Pompa Nyala, Estimasi 170.0s	PASSED
Level Aman	500.0	Pompa Standby	PASSED
Tangki Penuh	1000.0	Pompa Mati	PASSED
Error Handling	-50.0	Pesan Error (Level Negatif)	PASSED
Error Handling	1200.0	Pesan Error (Melebihi Kapasitas)	PASSED
Error Handling	abc	Pesan Error (Bukan Angka)	PASSED

Penjelasan pengujian

Berdasarkan hasil di atas, sistem dinyatakan **Lulus Uji (Passed)** karena mampu menangani skenario normal maupun kondisi anomali input tanpa mengalami kegagalan sistem (*crash*). Hal ini sesuai dengan kriteria keamanan instrumen industri yang membutuhkan reliabilitas tinggi.

9 Analisis Hasil

Fitur penyimpanan nilai *error* bekerja dengan baik. Sekalipun user berkali-kali memasukkan nilai sensor, sistem tetap menggunakan nilai bias pertama sebagai

acuan koreksi. Hal ini meniru perilaku PLC di industri yang menyimpan konstanta kalibrasi pada register memorinya. Konversi tinggi ke volume berjalan akurat secara linear.

1. **Penerapan Konsep OOP yang Efektif:** Penggunaan `struct SistemPompa` berhasil memodelkan aset fisik industri ke dalam bentuk digital secara terstruktur. Seluruh atribut seperti `kapasitas_tangki` dan `debit pompa` terbungkus rapi bersama metode operasionalnya (`hitungestimasiwaktu`) dalam blok `impl`, sesuai dengan prinsip enkapsulasi.

2. **Akurasi Komputasi Numerik:** Program mengimplementasikan perhitungan estimasi waktu pengisian secara linear yang akurat. Dengan menggunakan formula:

$$t = \frac{V_{kapasitas} - V_{level}}{Q_{debit}} \quad (2)$$

Sistem dapat memberikan informasi prediktif yang krusial bagi operator untuk mencegah *dry running* atau kerusakan peralatan.

3. **Validasi Input yang Robust:** Fungsi `minta_input_angka` menerapkan mekanisme penanganan kesalahan (*error handling*) yang kuat menggunakan `loop` dan *pattern matching* (`match`). Hal ini memastikan bahwa sistem hanya menerima angka desimal positif dan memberikan peringatan langsung jika input melebihi kapasitas tangki, sehingga meningkatkan keamanan operasional program.
4. **Logika Kontrol Interlocking:** Penggunaan struktur kendali `if-else` mencerminkan logika *interlocking* otomatis di industri. Sistem secara cerdas menentukan tiga status kritis:

- **TANGKI PENUH:** Mematikan pompa saat mencapai kapasitas maksimum untuk mencegah *overflow*.
- **LEVEL RENDAH:** Mengaktifkan pompa secara otomatis saat level di bawah ambang batas (*Low Level*).
- **LEVEL AMAN:** Menjaga sistem dalam kondisi siaga (*standby*).

5. **Keamanan Memori dan Performa:** Dengan basis bahasa Rust, program ini memiliki jaminan keamanan memori tanpa *garbage collector*, yang sangat penting untuk aplikasi sistem kontrol yang membutuhkan stabilitas tinggi dan penggunaan sumber daya yang efisien[cite: 30, 326].

10 Kesimpulan

Pengembangan sistem kontrol level tangki berbasis Rust ini membuktikan bahwa:

1. Konsep *Object Oriented Programming* (OOP) sangat efektif untuk memodelkan aset fisik seperti tangki dan pompa industri.

2. Logika *safety interlocking* yang diterapkan berhasil menjamin bahwa aktuator akan merespons sesuai dengan kondisi level yang telah dianalisis.
3. Rust mampu menghadirkan keseimbangan antara performa tinggi dan keamanan tipe data untuk pengembangan aplikasi instrumentasi industri.